Reproducing Confidential LLM Inference: A Performance Study of CPU Trusted Execution Environments

Raisson Adrian Grangeiro Souto
Universidade Federal de Campina Grande
Campina Grande, Paraíba, Brazil
raisson.souto@copin.ufcg.edu.br

Abstract

Large Language Models (LLMs) are increasingly deployed on cloud infrastructure where they process sensitive and proprietary data, raising significant concerns about model intellectual property theft and user data confidentiality. Trusted Execution Environments (TEEs) have emerged as a practical mechanism for end-to-end confidential LLM inference, offering hardware-enforced memory isolation with manageable performance overhead. This paper presents a reproduction study of Chrapek et al. [7], which originally characterized LLM inference performance and cost across CPU and GPU TEEs. We focus on reproducing the CPU TEE results for the Llama2 7B model, comparing Intel Software Guard Extensions (SGX) via the Gramine library OS and Intel Trust Domain Extensions (TDX) against a conventional virtual machine (VM) baseline. Our experimental design is multifactorial with 30 repetitions per configuration, varying batch sizes (1 and 64) and input sequence lengths (128, 512, and 2048 tokens), with Advanced Matrix Extensions (AMX) acceleration disabled to isolate the overhead contribution of the TEE mechanism itself. Two claims are under examination: whether CPU TEEs impose less than 10% throughput overhead on LLM inference on hardware distinct from the original study, and whether that relative overhead diminishes as batch size and input sequence length increase, as the amortization trend reported in the original study predicts.

CCS Concepts

• **Security and privacy** → **Trusted computing bases**; • **Computing methodologies** → *Machine learning*; • **Information systems** → *Cloud computing*.

Keywords

Confidential Computing; Trusted Execution Environments; Large Language Models; Intel SGX; Intel TDX

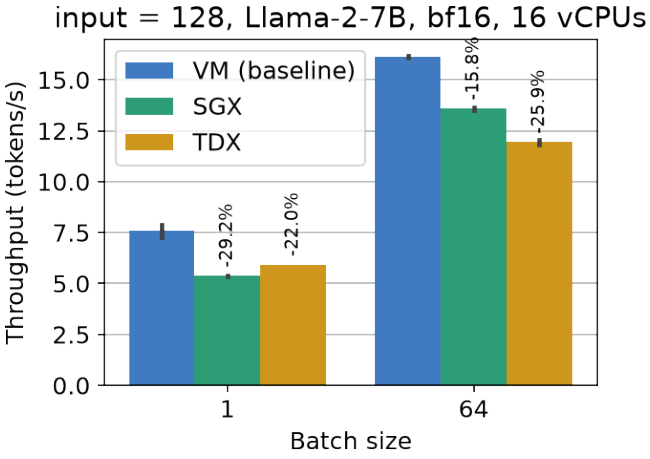

Figure 1: Median throughput at 128-token inputs, by batch size and execution environment.

1 Introduction

Large Language Models (LLMs) have become essential infrastructure across industries ranging from healthcare to financial analysis [17], yet their computational demands make on-premises deployment impractical for most organizations. LLM workloads are consequently migrated to Cloud Service Providers, where they routinely handle sensitive data: patient records, confidential legal documents, and proprietary source code [7]. This exposes them to threats from malicious insiders, adversarial co-tenants, and compromised hypervisors, with breaches carrying severe financial consequences given that model training costs can reach tens of millions of dollars [7].

The security research community has explored three primary classes of protection mechanisms for LLM deployments in adversarial cloud environments. *Machine learning-based methods*, including model watermarking and ownership verification schemes [5], operate post-hoc, detecting IP theft after it has occurred rather than actively preventing unauthorized access. These methods also fail to protect the confidentiality of user prompts. *Cryptographic methods*, such as Homomorphic Encryption (HE) [1] and Secure Multi-Party Computation (MPC) [13], provide strong provable security guarantees but impose performance overheads of up to four orders of magnitude, making them intractable for the compute-intensive inference workloads of modern LLMs. *Confidential Computing* via Trusted Execution Environments (TEEs) addresses both limitations: it actively enforces trust boundaries through hardware isolation and provides cryptographic attestation, with overheads substantially lower than cryptographic alternatives [12].

TEEs establish a *secure enclave*, a hardware-protected memory region inaccessible to the operating system, hypervisor, or co-tenants, and support *remote attestation*, allowing remote parties to verify the integrity and authenticity of the executing software. Two dominant strategies exist for CPU-based TEEs in cloud deployments. *Process-based TEEs*, exemplified by Intel Software Guard Extensions (SGX) [9], protect individual processes within an encrypted Enclave Page Cache (EPC), with overhead arising from EPC paging, enclave transitions, and memory integrity verification. *VM-based TEEs*, exemplified by Intel Trust Domain Extensions (TDX) [6] and AMD Secure Encrypted Virtualization (SEV-SNP) [2], protect an entire virtual machine using a hardened KVM hypervisor, offering

a broader trust boundary and simpler deployment at the cost of an additional virtualization performance penalty.

Chrapek et al. [7] conducted the first comprehensive study of LLM inference performance and cost across both CPU and GPU TEEs. Their evaluation runs full Llama2 (7B, 13B, 70B) inference pipelines within Intel SGX and TDX on two Intel Xeon server platforms, using Intel Extension for PyTorch (IPEX) [10] as the best-performing inference framework. A complementary GPU TEE evaluation is conducted on an NVIDIA H100 NVL instance from Microsoft Azure. From this evaluation, the authors derive 12 key insights. Among the most significant: CPU TEEs impose throughput overheads below 10% and latency overheads below 20%, further mitigated by Advanced Matrix Extensions (AMX) acceleration. TDX is considerably easier to deploy than SGX but incurs a higher virtualization tax of 1–5% on top of the baseline overhead, and for small LLMs at low batch sizes, CPU TEEs can be more cost-effective than confidential GPU counterparts.

Reproducibility is a cornerstone of scientific progress, yet independent re-execution of published computer-systems experiments remains rare and often difficult [8], and systems research results are frequently sensitive to hardware microarchitecture, software versions, and infrastructure configuration. This paper presents an independent reproduction of the CPU TEE results of Chrapek et al. [7]. Starting from the artifacts released by the authors,[1] we re-execute their measurement pipeline on public cloud infrastructure and assess whether the reported overhead bounds hold beyond the specific platforms on which they were established. The evaluation compares Intel SGX via the Gramine library OS and Intel TDX against a conventional VM baseline for Llama2 7B, across batch sizes of 1 and 64 and input lengths of 128, 512, and 2048 tokens, with 30 repetitions per configuration.

1.1 Research Questions

This reproduction study is guided by the following research questions:

- **RQ1:** Do CPU TEEs (Intel SGX, Intel TDX) impose less than 10% throughput overhead relative to a non-TEE VM baseline for Llama2 7B inference on independent hardware?
- **RQ2:** Does the relative throughput/latency overhead of CPU TEEs diminish as batch size and input sequence length increase?

RQ1 tests whether the original study's central overhead bound generalizes beyond the platforms on which it was established. The reproduction hardware is also Intel Xeon based, but differs in processor model, core count, and deployment model: 16-vCPU virtualized Azure instances rather than the dedicated Xeon Gold 6530 and Platinum 8580 servers used by Chrapek et al. RQ2 tests whether the overhead-amortization trend reported in the original work reproduces in this environment. Section 2 describes the experimental methodology, Section 3 reports how the collected evidence answers RQ1 and RQ2, Section 4 examines the threats to the validity of these findings, and Section 5 summarizes the study's conclusions and outlines directions for future work. Section 6 documents the published artifacts and how to reproduce this study, and a closing AI Usage Disclosure details the use of AI assistance in this work.

[1] https://github.com/spcl/confidential-llms-in-tees

Table 1: Experimental factors and their levels.

Factor	Levels
Execution environment	VM (baseline), SGX, TDX
Batch size	1, 64
Input sequence length	128, 512, 2048 tokens

2 Methodology

2.1 Scope

This study re-executes the experiment of Chrapek et al. [7] using a fork of their released codebase that fixes small bugs found during the reproduction,[2] focusing on CPU TEE results for **Llama2 7B** [14] across three execution environments: a conventional VM (baseline), Intel SGX via Gramine [15], and Intel TDX. GPU TEE experiments are not reproduced, as the CPU results constitute the primary empirical contribution of the original study. Batch sizes of 1 and 64 and input lengths of 128, 512, and 2048 tokens are selected to span the behavioral regimes identified in the original work. AMX acceleration is disabled throughout to isolate baseline TEE overhead, and all experiments use bfloat16 precision.

2.2 Experimental Design

The study follows a **multifactorial design** across execution environment, batch size, and input sequence length. Table 1 summarizes the factors and levels.

Among these factors, *batch size* deserves a brief explanation: it is the number of input sequences processed concurrently in a single inference pass. Larger batches amortize the fixed memory-bandwidth cost of streaming model weights per step across more sequences, raising throughput (tokens/s) at the cost of latency and memory (the KV cache alone grows linearly with batch size, beam width, and sequence length [11]). The two levels studied here thus capture the two canonical serving regimes: batch size 1 for interactive, latency-sensitive serving, and batch size 64 for throughput-oriented batch serving.

This yields a planned experimental matrix of 18 configurations ($3 \times 2 \times 3 = 18$), each with 30 independent repetitions (540 runs total). As discussed in Section 3.3, some configurations could not be completed because of memory limitations. Output length is fixed at 128 tokens, with greedy decoding at batch size 1 and 4-beam search at batch size 64, following the benchmark defaults of the original artifacts. Overheads are computed relative to the VM baseline within each factor combination.

Independent variables: execution environment (VM, SGX, TDX), batch size (1, 64), and input sequence length (128, 512, 2048 tokens).

Dependent variables: throughput (tokens/s) and next-token latency (ms), which directly instrument the research questions from Section 1.1: expressed as overhead relative to the VM baseline, they instrument **RQ1**, and their variation across the batch-size and input-length factor levels instruments **RQ2**. Medians across repetitions are reported for both metrics, together with 95% confidence intervals obtained by percentile bootstrap (10,000 resamples per

[2] https://github.com/raissonsouto/confidential-llms-in-tees/tree/develop

Table 2: Azure VMs used in this reproduction.

	SGX VM	TDX VM
Size	`Standard_DC16s_v3`	`Standard_DC16es_v6`
Family	DCsv3	DCesv6
vCPUs	16	16
Memory	128 GiB	64 GiB
CPU	Ice Lake (3rd gen)	Emerald Rapids (5th gen)
TEE	Intel SGX (64 GiB EPC)	Intel TDX
OS	Ubuntu 24.04 LTS (Gen2)	Ubuntu 24.04 LTS (CVM)
Serves as	VM baseline, SGX	TDX

cell). Each TEE is compared against the VM baseline at the same batch-size/input-length cell with a two-sided Mann-Whitney U test, which does not assume normality and suits the repetition counts used here ($n \approx 29$ per cell after discarding the warmup repetition).

2.3 Infrastructure and Software Stack

All experiments run on two Microsoft Azure VMs from Intel-based confidential-computing families, summarized in Table 2. The SGX machine is a `Standard_DC16s_v3` (DCsv3 family): 16 vCPUs on 3rd-generation Intel Xeon (Ice Lake) processors, 128 GiB of RAM, and a 64 GiB Enclave Page Cache (EPC). The TDX machine is a `Standard_DC16es_v6` (DCesv6 family) Confidential VM: 16 vCPUs on 5th-generation Intel Xeon (Emerald Rapids) processors and 64 GiB of RAM. Both run Ubuntu 24.04 LTS (the standard Generation 2 image on the SGX machine, the confidential-VM image on the TDX machine).

The conventional-VM baseline shares the SGX machine rather than occupying a third VM, because SGX is opt-in per process: `gramine-sgx` enters the enclave only for the workload it launches, so the same machine runs the same docker image with and without the TEE, and the SGX overhead is measured on identical hardware by construction. TDX, in contrast, encrypts the entire VM and cannot be disabled from inside a Confidential VM, so the TDX arm requires its own machine. The memory difference between the two VMs (128 vs. 64 GiB, a consequence of the DCsv3 family's fixed 1:8 vCPU-to-memory ratio) does not affect the batch-1 measurements: Llama2 7B in bfloat16 (≈14 GB) fits comfortably in both VMs and within the SGX EPC, avoiding EPC paging. At batch size 64, however, memory becomes the binding constraint: the TDX VM's 64 GiB (like the 64 GiB SGX enclave) is insufficient for 512-token inputs, and 2048-token inputs exceed even the 128 GiB machine (Section 3.3).

The software stack follows the original study [7]: IPEX [10] for inference (2.3 on the baseline and TDX arms, and 2.2 on the SGX arm, whose Gramine-enabled image is built on that release), PyTorch 2.x with Hugging Face Transformers 4.x [16] for model loading, Gramine [15] as the SGX runtime, and TCMalloc as the memory allocator. The baseline and TDX arms are driven by the DeepSpeed launcher [3], while the SGX arm uses the artifacts' single-instance execution path inside the enclave. Both paths run the same IPEX-optimized inference loop. Llama2 7B is loaded in bfloat16 without quantization.

2.4 Differences from the Original Study

Table 3 summarizes the main differences between the original study and this reproduction. The narrower scope reflects the available resources, the monetary cost of executing the full experimental matrix, and time constraints, rather than methodological disagreement. The selected configurations are sufficient to evaluate the original study's central performance claims.

The fork (Section 2.1) also diverges from the released artifact repository beyond the bug fixes summarized in Section 3.4. It expands the documentation (`AZURE.md` for VM provisioning, a rewritten end-to-end `README.md`, and notes on SGX pitfalls such as enclave sizing and offline model caching); publishes the raw experimental data alongside the code (per-run logs, hardware snapshots, and `results.csv`, letting future reproductions compare against both studies, Section 6); and adds tooling the original lacks (an SGX sweep driver, an Azure Retail Prices query script, a `config.env`

Table 3: Comparison between the original study and this reproduction.

Aspect	Original Study	This Reproduction
Models	Llama2 7B/13B/70B, Llama3 8B, Mistral 7B, Gemma 7B	Llama2 7B only
Execution environments	Bare metal, VM, Intel SGX, Intel TDX, NVIDIA H100 cGPU	VM (baseline), Intel SGX, Intel TDX (no bare metal, no GPU TEE)
Batch sizes	1–512	1, 64
Input lengths	32–2048 tokens	128, 512, 2048 tokens
AMX acceleration	Enabled and disabled (comparative analysis)	Disabled only
Data types	bfloat16 and int8	bfloat16 only
Hardware	Intel Xeon Gold 6530 (32 cores) and Intel Xeon Platinum 8580 (60 cores)	Azure `DC16s_v3` (Ice Lake, 16 vCPUs, 128 GiB) for baseline and SGX; Azure `DC16es_v6` (Emerald Rapids, 16 vCPUs, 64 GiB) for TDX
Repetitions	50 per configuration	30 per configuration
GPU TEE evaluation	NVIDIA H100 NVL (Azure, ≈$30K/month)	Not included

template, and pinned submodules), while dropping components outside this reproduction's scope (the GPU and RAG benchmark trees and the INT8 quantization scripts).

3 Results and Discussion

This section presents the results of reproducing the CPU TEE experiments of Chrapek et al. [7]. Throughput and next-token latency are analyzed to answer the research questions introduced in Section 1.1. Section 3.1 evaluates whether CPU TEEs remain below the 10% throughput overhead reported in the original study (RQ1), Section 3.2 examines whether the relative overhead decreases with increasing batch size and input length (RQ2), Section 3.3 discusses configurations that could not be executed because of memory limitations, and Section 3.4 summarizes the main findings.

3.1 Throughput and Latency Overhead (RQ1)

Table 4 reports, for each of the 18 configurations (3 execution environments × 2 batch sizes × 3 input lengths, 30 repetitions each), the median throughput, median next-token latency, and the resulting overhead relative to the VM baseline within the same batch size/input length cell. Figure 1 shows the median throughput at 128-token inputs, by batch size and execution environment.

The evaluated configurations do not reproduce the throughput overhead bounds reported in the original study for either TEE. For TDX, throughput overhead relative to the VM baseline ranges from 25.3% [95% CI 24.7, 25.8] to 44.5% [44.4, 44.7] across the four measurable configurations (Table 4), well above the 10% bound reported by Chrapek et al. [7]. Next-token latency shows the same picture (e.g., 154.0 vs. 112.2 ms at batch 1 with 128-token inputs, +37%, against the original study's <20% latency bound, $p < 10^{-6}$). Part of the TDX gap may be attributable to the platform rather than to TDX itself, since that arm runs on a different machine and CPU generation than the baseline (Section 4). The SGX arm, however, settles the question without that confound: measured against the baseline on the very same machine, SGX still exceeds the bound, with 32.4% [32.1, 32.6], 30.7% [30.5, 30.8], and 25.8% [25.6, 26.5] throughput overhead at batch 1 for 128-, 512-, and 2048-token inputs, 15.5% [14.7, 16.4] at the single batch-64 cell that completed (all $p < 10^{-6}$), and next-token latency overhead around +54% at batch 1 (172.3 vs. 112.2 ms at 128 tokens).

3.2 Effect of Batch Size and Input Length (RQ2)

The two TEEs diverge. For TDX, relative throughput overhead *grows* with input length at batch size 1 (25.5% [25.3, 25.7] at 128 tokens, 34.1% [33.9, 34.1] at 512, and 44.5% [44.4, 44.7] at 2048, all $p < 10^{-6}$ against the baseline), and the single measurable batch-64 cell (128-token inputs, 25.3% [24.7, 25.8]) sits at the same level as its batch-1 counterpart rather than below it. On this evidence, the overhead-amortization trend does not reproduce for TDX in this environment. For SGX, in contrast, overhead *shrinks* with input length (32.4% [32.1, 32.6] → 30.7% [30.5, 30.8] → 25.8% [25.6, 26.5]) and drops to 15.5% [14.7, 16.4] at the single measurable batch-64 cell, less than half its batch-1 counterpart, which is consistent with the original study's amortization claim (Figures 9–10 of Chrapek et al. [7]), although the levels remain well above the original bounds. Figure 2 consolidates both trends across the full factor space.

Two methodological gaps likely explain why TDX fails to reproduce this trend while SGX does not. First, Chrapek et al.'s amortization argument reflects compute-boundedness: their input-length sweep (Figure 10) runs at batch size 64, already AMX-saturated, and even there the trend reverses at the longest inputs once the growing KV cache starts missing cache. This reproduction's batch-1 measurements lack the batching needed to raise arithmetic intensity in the first place, so that shrinking-overhead mechanism may simply not engage. Second, only SGX is measured confound-free:

Table 4: Median throughput, latency, and overhead relative to the VM baseline, by configuration, with 95% bootstrap confidence intervals; OOM: out of memory.

Environment	Batch	Input (tok)	Throughput (tok/s)	Latency (ms)	Overhead (%)
VM (baseline)	1	128	7.96 [7.94, 7.98]	112.2 [111.9, 112.4]	—
		512	6.87 [6.86, 6.88]	116.9 [116.8, 117.2]	—
		2048	4.11 [4.10, 4.12]	126.0 [125.1, 126.4]	—
	64	128	16.15 [16.01, 16.25]	3566.0 [3541.2, 3598.8]	—
		512	10.88 [10.86, 10.89]	4193.4 [4186.7, 4203.2]	—
		2048		OOM	
SGX (Gramine)	1	128	5.38 [5.37, 5.40]	172.3 [171.7, 172.6]	32.4 [32.1, 32.6]
		512	4.76 [4.76, 4.78]	177.1 [176.5, 177.4]	30.7 [30.5, 30.8]
		2048	3.05 [3.02, 3.06]	190.5 [189.6, 193.5]	25.8 [25.6, 26.5]
	64	128	13.64 [13.51, 13.71]	4211.3 [4187.9, 4257.3]	15.5 [14.7, 16.4]
		512		OOM	
		2048		OOM	
TDX	1	128	5.93 [5.92, 5.93]	154.0 [153.9, 154.2]	25.5 [25.3, 25.7]
		512	4.53 [4.53, 4.53]	158.3 [158.2, 158.4]	34.1 [33.9, 34.1]
		2048	2.28 [2.28, 2.28]	173.8 [173.7, 173.9]	44.5 [44.4, 44.7]
	64	128	12.06 [12.06, 12.07]	4358.7 [4356.8, 4361.5]	25.3 [24.7, 25.8]
		512		OOM	
		2048		OOM	

the TDX-vs-baseline comparison spans two CPU generations (Section 4), so a growing Ice Lake/Emerald Rapids gap with sequence length could masquerade as rising "TDX overhead" even if the confidential-computing tax itself were flat.

3.3 Infeasible Configurations at Batch Size 64

Five of the eighteen configurations proved infeasible, all at batch size 64. On the TDX VM (64 GiB), the 512-token configuration was terminated by the Linux kernel's out-of-memory (OOM) killer on two independent attempts, both ≈15 minutes into the run during the prefill of the first warmup iterations, with the inference process holding ≈59 GiB of anonymous resident memory. On the 128 GiB baseline machine, the 2048-token configuration was likewise OOM-killed (≈123 GiB resident, of ≈187 GiB of requested virtual memory), which rules out that input length for all three environments at this batch size: the TDX VM is strictly smaller, and the SGX arm runs on the same 128 GiB host. The corresponding cells of Table 4 are marked OOM. The 512-token configuration did complete on the baseline machine. Under SGX it faces a third, distinct memory ceiling: the Gramine manifest sizes the enclave to the 64 GiB EPC, and the 512-token runs at batch size 64 failed with allocation errors inside the enclave (`DefaultCPUAllocator: can't allocate memory`), while the 128-token runs completed.

This infeasibility is a reproduction finding in its own right. The model itself (≈14 GB in bfloat16) is not the constraint: at batch size 64 the benchmark uses 4-beam search, so the KV cache and prefill activation buffers scale with batch × beams × sequence length and dominate memory consumption. That the 512-token case fits in 128 GiB but not in the TDX VM's 64 GiB follows directly from Azure's confidential-VM sizing: the DCesv6 family provides a fixed 1:4 vCPU-to-memory ratio (16 vCPUs → 64 GiB), versus 1:8 for DCsv3. The practical consequence is that the large-batch, long-input regime in which the original study reports the strongest overhead amortization (with batch sizes up to 512) is not reachable at the VM sizes used in this reproduction.

3.4 Key Findings

- **RQ1:** the <10% bound on throughput overhead does *not* hold in this reproduction (all $p < 10^{-6}$). TDX shows 25.3–44.5% (subject to the confound of two CPU generations, Section 4) and SGX, measured on the same machine as its baseline and therefore confound-free, shows 25.8–32.4% at batch 1 and 15.5% at batch 64.
- **RQ2:** the TEEs diverge. TDX overhead *increases* with input length at batch 1 (25.5% → 44.5%) and does not decrease at batch 64, contradicting the amortization trend of the original study, while SGX overhead decreases with input length (32.4% → 25.8%) and with batch size (15.5% at batch 64), consistent with it. This divergence tracks the study's confound: SGX is measured confound-free, TDX is not, and the original amortization argument is tested at batch 64, a regime already compute-saturated on the AMX units, not at batch 1.
- **Feasibility:** at batch size 64, memory is the binding constraint: 512-token inputs exceed the TDX CVM's 64 GiB and the 64 GiB SGX enclave, and 2048-token inputs exceed even

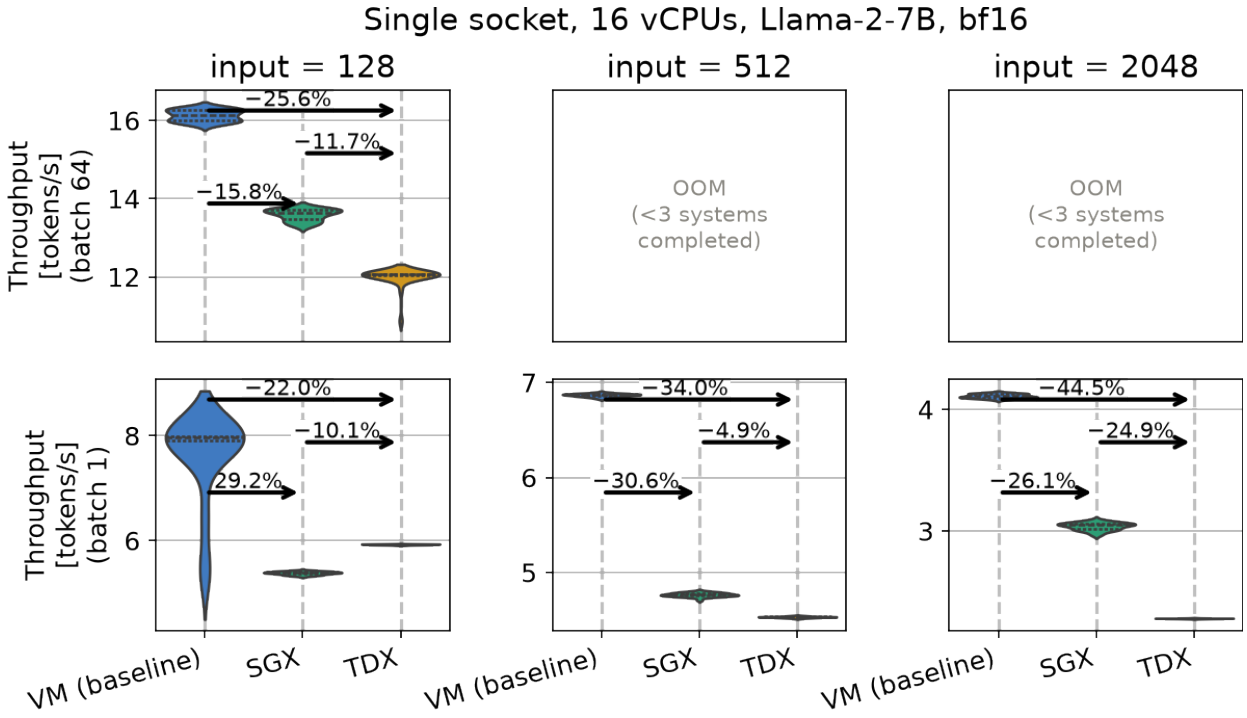

Figure 2: Throughput distributions by environment, input length (columns), and batch size (rows), annotated with median relative differences. Batch-64 panels at 512 and 2048 tokens are marked OOM (Section 3.3).

the 128 GiB baseline/SGX machine (Section 3.3). The large-batch factor space of the original study is not fully reachable at these Azure VM sizes.

4 Threats to Validity

Measurements are taken on shared, virtualized cloud infrastructure (Microsoft Azure), so noisy-neighbor effects from co-located tenants and CPU frequency/turbo-boost variability across repetitions could inflate variance independent of the TEE mechanism under test. This is mitigated by taking 30 repetitions per configuration and reporting medians, and by pinning the same 16-vCPU core binding everywhere and the same Docker image (`ipex-llm:2.3.100`) on the baseline and TDX arms. The SGX arm cannot share that image: following the original artifacts, it runs the Gramine-enabled `sgx-ipex-llm:2.2.0` image, built on IPEX 2.2 rather than 2.3, a software-version difference that partially confounds the SGX-vs-baseline comparison. A residual confound remains for TDX: because TDX cannot be disabled from inside a Confidential VM, its overhead is computed against the VM baseline measured on the Ice Lake SGX machine, so the TDX-vs-baseline comparison spans two CPU generations (Table 2). The SGX-vs-baseline comparison is free of this confound, since both arms share the same machine. The TDX VM's smaller memory additionally truncates the TDX factor space at batch 64 (Section 3.3), so the batch-size/input-length interaction can only be compared across all three environments for the configurations that fit in 64 GiB.

The reproduction hardware (the two 16-vCPU Azure VMs of Table 2) differs from the original study's Intel Xeon Gold 6530 and Platinum 8580 server platforms, which may have different SGX EPC sizes, TDX virtualization stacks, and AMX/AVX-512 microarchitectural behavior even with AMX disabled. The factor space is also narrower than the original study's, as summarized in Table 3: only Llama2 7B is evaluated (no 13B/70B, Llama3, Mistral, or Gemma), no bare-metal baseline is included, and GPU TEEs are out of scope. Conclusions drawn here therefore generalize to this specific model size and hardware class, not to the full space covered by the original study.

Throughput and next-token latency are used as proxies for "performance overhead," following the original study's definition. Medians are reported alongside bootstrap confidence intervals and full throughput distributions (Figure 2), but tail percentiles (e.g., P99 latency) that matter for latency-sensitive deployments are not. The synthetic, fixed-length workloads (128/512/2048 input tokens, 128 output tokens, greedy decoding) may not reflect the input-length distribution or decoding strategy of production LLM serving traffic.

Medians are accompanied by bootstrap 95% confidence intervals, and every TEE-vs-baseline comparison is tested with a two-sided Mann-Whitney U test ($p < 10^{-6}$ throughout, Table 4), so the RQ1/RQ2 conclusions rest on statistical inference rather than on point-estimate magnitude alone. The small p-values mostly reflect the low measurement noise of dedicated, non-shared VMs and the resulting non-overlapping repetition distributions, not necessarily large practical effect sizes at the margins (e.g., the 15.5% vs. 25.3% batch-64 overheads for SGX and TDX).

5 Conclusion

This paper reproduced the CPU Trusted Execution Environment (TEE) evaluation of Chrapek et al. [7] on independent Azure hardware, comparing Intel SGX (via Gramine) and Intel TDX against a conventional VM baseline for Llama2 7B inference. The results indicate that the original study's reported throughput overhead bound of less than 10% does not reproduce in the evaluated environment. Measured throughput overhead ranges from 15.5–32.4% for SGX and from 25.3–44.5% for TDX, every cell statistically significant against the baseline (Mann-Whitney U, $p < 10^{-6}$; Table 4). Furthermore, the overhead amortization trend reported in the original work is reproduced for SGX but not for TDX under the evaluated configurations.

The reproduction also revealed practical deployment limitations that were not evident from the original study. Several large-batch configurations exceeded the available memory of the Azure confidential VM instances, preventing the evaluation of part of the experimental matrix. In addition, reproducing the experiments required correcting issues in the released artifacts, including repository configuration and execution scripts, which are documented in the accompanying fork.

The evidence collected so far supports the following key findings:

- **TEE overheads far exceed the original bounds.** Measured throughput overhead is 25.3–44.5% for TDX and 15.5–32.4% for SGX (vs. the original <10%; all $p < 10^{-6}$), with next-token latency overheads at batch 1 of roughly +54% (SGX) and +37% (TDX) against the original <20%. The SGX result is measured on the same machine as its baseline and is therefore free of the cross-generation confound of Section 4.
- **Amortization reproduces only for SGX.** At batch size 1, TDX overhead increases monotonically with input length (25.5% → 34.1% → 44.5%), contradicting the amortization trend of the original study, while SGX overhead decreases with input length (32.4% → 30.7% → 25.8%) and with batch size (15.5% at batch 64), consistent with it. The original study's amortization argument is tested at batch 64, an already compute-saturated regime; the TDX-vs-baseline comparison also spans two CPU generations (Section 4), so its divergence is not attributable to TDX alone.
- **Memory, not compute, bounds the factor space.** At batch size 64 the benchmark's 4-beam search makes memory the binding constraint, with three distinct ceilings (the TDX CVM's 64 GiB, the baseline host's 128 GiB, and the 64 GB SGX enclave), rendering the original study's large-batch regime unreachable at these Azure VM sizes.
- **The released artifacts required fixes to reproduce.** Missing submodule gitlinks, a non-recursive glob in the results parser, and a whitespace-corrupted patch that broke the SGX+IPEX path all had to be repaired. The fixes are collected in the fork cited in Section 2.1.

Overall, these results reinforce the importance of independent reproduction studies in systems research. They show that the performance of confidential LLM inference is sensitive to hardware generation, virtualization environment, software stack, and available memory resources. Future work includes extending the evaluation

to GPU TEEs, larger language models, and additional cloud platforms, as well as running the original study's input-length sweep at batch size 1 to test directly whether the compute-boundedness argument behind TDX's amortization claim (Section 3.2) holds outside the batch-64 regime in which it was originally measured.

6 Reproducibility

All artifacts of this reproduction are public, following the availability criteria of the ACM artifact review and badging policy [4], in a fork of the original study's repository.[3]

What is available. The fork contains the benchmark scripts and patches (with third-party dependencies pinned as submodules), `AZURE.md` and `README.md` with end-to-end setup and run instructions, the `results/` directory with raw per-arm logs, hardware snapshots, and the consolidated `results.csv`, the figures of Section 3 with the plotting scripts that produce them, `overhead_stats.py` for the bootstrap confidence intervals and Mann-Whitney U tests of Table 4, and the LaTeX source of this paper in `paper/`. Failed runs (e.g., out-of-memory at batch 64) are published as-is, since the failure modes are themselves findings; the original study's repository, by contrast, releases only the benchmark code, so publishing the measurement data here lets future reproductions compare directly against both studies.

How to reproduce. (1) Provision the two VMs following `AZURE.md`; (2) on each VM, follow the `README.md` common setup (clone, initialize submodules, apply patches, run `host_setup.sh`, and build the Docker image); (3) run the baseline and TDX arms with `./run.sh baseline` and `./run.sh tdx`, and the SGX arm with `sgx_setup.sh` followed by `run_sgx_sweep.sh` (from `CPU/`); (4) parse the logs with `run_parser.py` to obtain the CSV used in the analysis, and run `overhead_stats.py` on it to reproduce Table 4.

Required resources. Two Azure VMs: `Standard_DC16s_v3` (16 vCPUs, 128 GB, Ice Lake; baseline and SGX) and `Standard_DC16es_v6` (16 vCPUs, 64 GB, Emerald Rapids; TDX), both on Ubuntu 24.04 with a 128 GB OS disk, plus vCPU quota for the `DCSv3`/`DCESv6` families (zero by default) and a Hugging Face token with the Llama-2 license accepted. At roughly US$1.5–2 per VM-hour, a full sweep per arm takes several hours, dominated by the batch-64 configurations.

AI Usage Disclosure

The author used Claude Code (Claude, Anthropic) as an assistant throughout this project: diagnosing and patching the original artifact's build and benchmark scripts, drafting the provisioning runbook, analyzing benchmark logs, generating the consolidated results CSV, and revising the documentation and the LaTeX source of this paper. All experimental design decisions, the execution of the experiments, and the validation of results and text remain the author's; the author reviewed all AI-assisted output and takes full responsibility for the content of this paper and its artifacts.

References

[1] Abbas Acar, Hidayet Aksu, A. Selcuk Uluagac, and Mauro Conti. 2018. A Survey on Homomorphic Encryption Schemes: Theory and Implementation. *Comput. Surveys* 51, 4 (2018), 79:1–79:35.

[2] Advanced Micro Devices, Inc. 2020. AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More. White paper. https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf.

[3] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, Dallas, TX, USA, 46:1–46:15.

[4] Association for Computing Machinery. 2020. ACM Artifact Review and Badging Policy. https://www.acm.org/publications/policies/artifact-review-and-badging-current.

[5] Franziska Boenisch. 2021. A Systematic Review on Model Watermarking for Neural Networks. *Frontiers in Big Data* 4 (2021), 729663.

[6] Po-Chang Cheng, Wojciech Ozga, Enrique Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. 2024. Intel TDX Demystified: A Top-Down Approach. *Comput. Surveys* 56, 9 (2024), 238:1–238:33.

[7] Marcin Chrapek, Marcin Copik, Etienne Mettaz, and Torsten Hoefler. 2025. Confidential LLM Inference: Performance and Cost Across CPU and GPU TEEs. arXiv:2509.18886 [cs.PF] https://arxiv.org/abs/2509.18886.

[8] Christian Collberg and Todd A. Proebsting. 2016. Repeatability in Computer Systems Research. *Commun. ACM* 59, 3 (2016), 62–69.

[9] Victor Costan and Srinivas Devadas. 2016. Intel SGX Explained. arXiv:1702.01432

[10] Intel Corporation. 2024. Intel® Extension for PyTorch. https://github.com/intel/intel-extension-for-pytorch. Accessed: 2026.

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, Koblenz, Germany, 611–626.

[12] Fan Mo, Zahra Tarkhani, and Hamed Haddadi. 2024. Machine Learning with Confidential Computing: A Systematization of Knowledge. *Comput. Surveys* 56, 11 (2024), 281:1–281:40.

[13] Payman Mohassel and Yupeng Zhang. 2017. SecureML: A System for Scalable Privacy-Preserving Machine Learning. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Jose, CA, USA, 19–38.

[14] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]

[15] Chia-Che Tsai, Donald E. Porter, and Mona Vij. 2017. Graphene-SGX: A Practical Library OS for Unmodified Applications on SGX. In *2017 USENIX Annual Technical Conference (USENIX ATC'17)*. USENIX Association, 645–658.

[16] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45.

[17] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A Survey of Large Language Models. arXiv:2303.18223

[3] https://github.com/raissonsouto/confidential-llms-in-tees/tree/develop